Data Models

Normalisation

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Topic 2

Topic 2 produced a **structurally correct schema** from an ERD — every cardinality captured, every participation constraint honoured.

But structural correctness is not the whole story. The ERD ensures the schema captures **business rules**. It does not check whether the same fact is stored in **two places**.

If one copy is updated and the other is not, **truth drifts** — and the database never objects.

ShopEasy: The City That Drifted

ShopEasy — the same e-commerce platform from Topic 2's payment cardinality ambiguity.

A developer stored the customer's city in both `customers` and `orders` — one fact in two places:

```
customers                                orders
customer_id | name       | city        order_id | customer_id | city
————————————|————————————|———————      —————————|————————————|———————
1           | Alice Tan  | Singapore      101   | 1          | Singapore
4           | Dave Wong  | Singapore      102   | 4          | Singapore
```

The reasoning: show the customer's city on the order record without a join.

For six months, this worked.

ShopEasy: Truth Drifts

Singapore's Urban Redevelopment Authority renamed three planning areas. The operations team updated `customers`. Nobody touched `orders`.

```
customers (updated)                orders (stale)
customer_id | city                 order_id | city
————————————|———————————————       —————————|———————————————
1           | Singapore Central    101      | Singapore
4           | Singapore Central    102      | Singapore
```

The "revenue by city" report returned **six rows** where three were expected. The financial controller flagged a **$140,000 discrepancy**.

Root cause: one editable fact — the mapping from postcode to city — stored in two places. Redundant mutable facts eventually diverge.

Cost: two-week investigation, migration across 1.2 million order records, months of lost trust in city-level reporting.

ShopEasy: The Dashboard Nobody Used

After the city incident, the team normalised aggressively. Product details split across `products`, `categories`, `departments`, `suppliers` — each fact in exactly one place.

Then the reporting team asked: "**total revenue by department by quarter by supplier region.**"

- **5 tables** joined, **4 joins**, **38 seconds** on 50,000 order lines
- Filters and drill-downs made the dashboard unusable
- The monthly reporting meeting reverted to a **manually assembled spreadsheet**

The schema was *correct* — every table in BCNF, every fact stored once. But the **shape was wrong for the workload**.

Two Failures, One Tension

	Failure 1: City drift	Failure 2: Slow dashboard
Symptom	Six rows where three expected; $140K discrepancy	38-second query; dashboard abandoned
Root cause	One editable fact in two places	Normalised schema serving analytical workload
Schema status	Not BCNF (redundancy)	BCNF (no redundancy)
Lesson	Redundancy corrupts truth	Full normalisation kills reporting

Predict: Failure 1 says "normalise." Failure 2 says "don't normalise." Are these contradictory?

The Resolution: Two Workloads, Two Shapes

They are not contradictory. They are two different **workloads** demanding two different **shapes**.

The resolution: **BCNF for transactional truth**, a **star schema for analytical reporting**, with the star *derived from* the normalised source.

This is Topic 1's **shape alignment** principle applied within the relational model — one data model, two shapes for two access patterns.

> Therefore, the designer must **eliminate redundancy for transactional truth and build a separate derived shape for reporting** — not choose between correctness and performance.

OLTP vs OLAP: Two Workloads

	OLTP (Transactional)	OLAP (Analytical)
Access pattern	Many small reads/writes; concurrent updates	Read-heavy aggregates over time/category/region
Edit model	Frequent inserts, updates, deletes	No direct edits
Design goal	Eliminate redundancy → one fact, one place	Optimise for query simplicity → deliberate denormalisation
Formal shape	Normalised (BCNF)	Star schema

The question that organises this topic: given a set of tables, how do you verify they are free of redundancy, name the violations, fix them, and then safely build the denormalised shape that reporting demands?

Three Capabilities

Each requirement maps to a failure:

Requirement	Failure Prevented	Test
Eliminate redundancy in editable facts — one fact, one place	Failure 1: city drifted across two tables	Every determinant in every OLTP table is a candidate key
Diagnostic vocabulary — name the kind of redundancy	Both: team could not name or classify the problem	Given a table and its FDs, state its normal form and the violating dependency
Derived reporting structures — star schema, read-only, rebuilt from normalised truth	Failure 2: normalised schema killed dashboard performance	Star schema query uses fewer joins, produces the same totals

What formal vocabulary do we need to diagnose redundancy, name normal forms, and describe reporting shapes?

Functional Dependency: What Determines What

In the `customers` table, `customer_id → name` means customer 1 is always 'Alice Tan'. This is what a primary key does — it determines everything else in the row.

But FDs are not limited to primary keys. In Failure 1, `postcode → city` is an FD: postcode 018956 determines city Singapore. The postcode is a **determinant** — but not the primary key. That mismatch is exactly what caused the drift.

> **Functional dependency (FD):** $X \rightarrow Y$ — if two rows agree on X, they must agree on Y. Knowing X uniquely determines Y.

FDs in Sample Data

`customer_id → name` holds — every `customer_id` value appears once:

```
customers
customer_id | name        | email                | postcode
------------|-------------|----------------------|----------
1           | Alice Tan   | alice@example.com    | 018956
2           | Bob Patel   | bob@example.com      | 50088
3           | Carol Lim   | carol@example.com    | 10110
4           | Dave Wong   | dave@example.com     | 018956
```

Also: `postcode → city` holds (if city were stored here). Alice and Dave share postcode `018956` — both must map to the same city. One fact, but if stored on this table, it would be recorded **twice**.

Composite Keys and FDs

The `order_lines` table has a composite key `(order_id, product_id)`:

```
order_lines
order_id | product_id   | quantity | unit_price
---------+--------------+----------+-----------
101      | 1 (Widget A)| 2        | 9.99
101      | 2 (Widget B)| 1        | 14.99
102      | 3 (Gadget X)| 1        | 49.99
103      | 1 (Widget A)| 1        | 9.99
```

Predict: Is `order_id → unit_price` a valid FD?

Composite Keys: Revealed

No. Order 101 appears in two rows with different `unit_price` values (9.99 and 14.99). The same `order_id` maps to different prices — the FD does not hold.

You need the **full composite key** `{order_id, product_id}` to determine `unit_price`. Each combination appears exactly once.

The lesson: with composite keys, check whether each part alone determines non-key attributes. If it does, you have a **partial dependency** — a normal form violation we will name shortly.

Candidate Key

FDs tell us what determines what. The next question: which determinants can serve as the row's **identifier**?

A **candidate key** is a minimal set of attributes that uniquely identifies every row. "Minimal" means removing any attribute breaks uniqueness.

- Most tables have one candidate key → becomes the **primary key**
- Some have multiple: `customers` might have both `customer_id` and `email` (if emails are unique)
- All candidate keys share one property: they **functionally determine every other attribute**

> **Candidate key:** a minimal set of attributes whose values uniquely determine every other attribute in the table.

The Normal Form Ladder

A **diagnostic** tool — not a construction procedure. It classifies tables by the *kind* of redundancy they contain:

Normal Form	Rule	What It Catches
1NF	Every cell atomic; no repeating groups	Multi-valued cells, repeated column groups
2NF	No non-key attribute depends on only *part* of a composite key	Descriptors duplicated across line items
3NF	No non-key attribute depends on another non-key attribute	Transitive chains like `postcode → city` (Failure 1)
BCNF	Every determinant is a candidate key	The strictest practical target — "one table, one fact"

Strict containment: **BCNF ⊂ 3NF ⊂ 2NF ⊂ 1NF**. A table in BCNF is automatically in 3NF, 2NF, and 1NF.

Not 1NF: Multi-Valued Cells

```
students
student_id | name     | courses
-----------+----------+-------------------------------
S1         | Mei Lin  | Math, Physics, Chemistry
S2         | Ravi     | Math, Biology
```

The `courses` column contains multiple values in a single cell — not atomic. The table is **not even in 1NF**.

Fix: create a separate `enrolments(student_id, course_id)` table — one row per student-course pair.

1NF but Not 2NF: Partial Dependency

```
enrolments (composite key: student_id, course_id)
student_id | course_id | student_name | course_title | grade
————————————|————————————|————————————————|————————————————|———————
S1          | C101       | Mei Lin       | Databases      | A
S1          | C102       | Mei Lin       | Networks       | B+
S2          | C101       | Ravi          | Databases      | B
```

`student_name` depends on `student_id` alone — **partial dependency**. Mei Lin's name is repeated for every course.

Fix: decompose into `students(student_id, student_name)`, `courses(course_id, course_title)`, `enrolments(student_id, course_id, grade)`.

2NF but Not 3NF: Transitive Dependency

```
customers (before decomposition)
customer_id | name        | postcode | city
------------|-------------|----------|------------
1           | Alice Tan   | 018956   | Singapore
2           | Bob Patel   | 50088    | Kuala Lumpur
3           | Carol Lim   | 10110    | Bangkok
4           | Dave Wong   | 018956   | Singapore
```

The chain: `customer_id → postcode → city`. The city depends on `postcode`, not on the customer ID. Alice and Dave both store `018956 → Singapore` — the **same fact recorded twice**.

This is exactly **Failure 1**. The table is in 2NF (single-column key, no partial dependencies) but not 3NF.

BCNF: Every Determinant Is a Key

BCNF is the strictest practical target. The rule: every determinant in the table is a candidate key. If a column determines another column, it must be a key — otherwise, you have a second path for the same fact to drift.

You do not *build* tables by walking 1NF → 2NF → 3NF → BCNF. You **design directly to BCNF**. The ladder exists for *diagnosis* — naming what is wrong with a legacy table.

> **Predict:** The `orders` table has `order_id`, `customer_id`, `order_date`, `status`. Does every non-key column describe the thing identified by `order_id`?

"One Table, One Fact": Revealed

Yes. `customer_id` identifies which customer placed *this order*. `order_date` is the date of *this order*. `status` is the status of *this order*. Every non-key column describes the order, not something else.

Formally: the only determinant (`order_id`) is a candidate key. **The table is in BCNF.**

The intuitive test: **does every non-key column describe the thing identified by the primary key, and *only* that thing?** If a column describes something else — a postcode describes a city, not a customer — it belongs in its own table.

The intuitive test and the formal BCNF definition are two views of the same principle.

Lossless Decomposition: The Safety Net

When you split a table to eliminate redundancy, the join must recover the original data exactly. If the join produces extra rows (**spurious tuples**), the decomposition is *lossy*.

The test: the shared columns must be a candidate key of at least one part.

```
SAFE:   (student_id, course_id) + (course_id, instructor)
         shared: course_id — key of second table ✓

LOSSY:  (student_id, instructor) + (course_id, instructor)
         shared: instructor — not a key of either ✗
         → phantom student-course combinations
```

In practice, decomposing along a functional dependency — which is what the normal form rules tell you to do — is always lossless.

The OLTP vocabulary is now complete — FDs, candidate keys, normal forms, and lossless decomposition. What about the reporting workload?

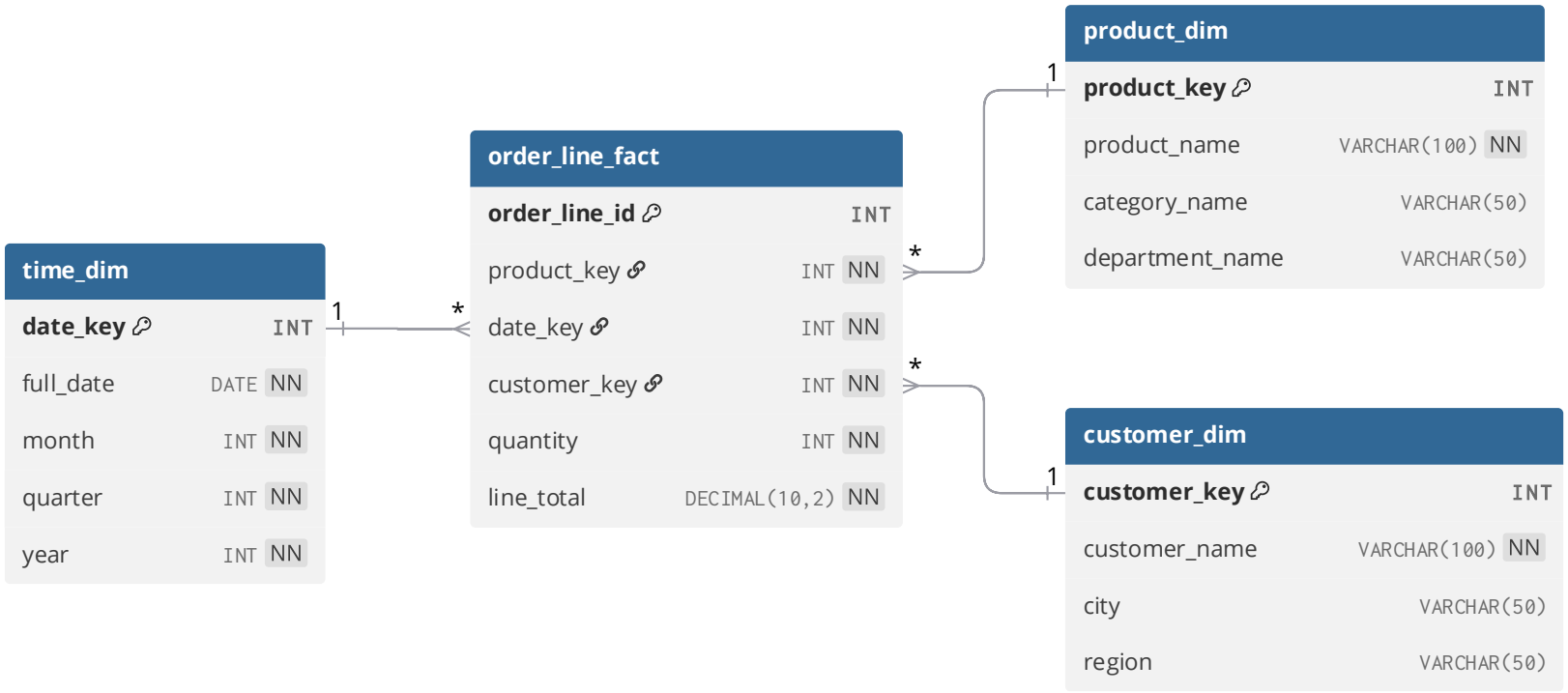
Star Schema: The OLAP Counterpart

The star schema serves the reporting workload. Three components:

Component	Role	Example
Fact table	One row per business event; holds measures at a defined *grain*	`order_line_fact(product_key, date_key, customer_key, quantity, line_total)`
Dimension table	Descriptive context for one axis; deliberately denormalised	`product_dim(product_key, product_name, category_name, department_name)`
Grain	What one row of the fact table represents	"One line item in one order"

The fact table sits at the centre; dimension tables radiate outward — **one join from fact to dimension, no chains**.

Star Schema: The Shape



`product_dim` has a transitive dependency: `product_key → category_name → department_name`. A deliberate **3NF violation**.

> **Predict:** What normal form is `product_dim` in?

product_dim Normal Form: Revealed

> The table is in **2NF but not 3NF**. The transitive dependency `category_name → department_name` means a non-key attribute determines another non-key attribute.

In an OLTP context, this would be a design flaw — the same violation that caused Failure 1.

In the star schema, it is the **whole point**. The flat dimension eliminates the join chain that made the dashboard unusable (Failure 2). The violation is acceptable because the star schema is:

- **Read-only** — no edits, no update anomalies
- **Derived** — rebuilt from the normalised BCNF source
- **Bounded** — only dimension tables contain redundancy

We have seen both shapes — BCNF and star. Why does each work for its workload?

Why BCNF Eliminates Truth Drift

If every determinant is a candidate key, every non-key fact has **one determination path** — through the key. One path → one copy → no drift.

In Failure 1, `postcode` determined `city` but was not a candidate key. That gave `city` **two paths** — and when `city` also appeared on `orders`, three places could hold the same fact with nothing forcing them to agree.

Decomposing into `customers( ... , postcode)` + `postcodes(postcode, city)` makes `postcode` the key of its own table. City stored **once**. The join is the cost; the safety is the benefit. BCNF is the **design-time discipline** that shapes the schema so the engine's runtime constraints (PK, FK, UNIQUE) are sufficient.

Why the Normal Form Ladder Matters

Each level names a different disease — and each disease has its own cure:

Level	Disease	Cure
Not 1NF	Multi-valued cells, repeating groups	Separate table, one row per value
1NF not 2NF	Partial dependency on composite key	Split out the partial determinant
2NF not 3NF	Transitive dependency (non-key → non-key)	Extract lookup table for the chain
3NF not BCNF	Non-key determinant exists	Decompose along the violating FD

Without the vocabulary: "this table is bad." With it: "2NF but not 3NF — transitive dependency `postcode → city` — decompose to a lookup table." **The diagnosis is the prescription.**

Why Full Normalisation Kills Reporting

BCNF distributes facts across many small tables. Each table is clean — but analytical queries must **reconstruct** the picture by joining them back.

Failure 2's "revenue by department by quarter" query:

```
order_lines → products → categories → departments → orders (for date)
    JOIN 1       JOIN 2      JOIN 3        JOIN 4
```

Four joins across five tables. Each join reconstructs what normalisation deliberately broke apart. The join chain is the structural **cost** of normalisation.

- **OLTP cost:** acceptable — transactional queries rarely need all five tables
- **OLAP cost:** fatal — *every* analytical query needs the full picture

Why Star Schema Denormalisation Is Safe

Three reasons the same 3NF violation that caused Failure 1 is safe in a star schema:

Read-only. Dimension tables are not edited by transactional users. No updates → no update anomalies. Nobody changes `department_name` in `product_dim` directly.

Derived from normalised truth. The star is rebuilt from the BCNF source via ETL. If the source changes, the star is regenerated — it is never the primary record.

```
OLTP (read/write, BCNF) ──▶ ETL ──▶ STAR (read-only, 2NF)
```

Controlled scope. The violations are named, documented, and bounded — only dimension tables contain redundancy.

> **Predict:** A department is renamed from "Home & Garden" to "Home & Living" in the normalised `departments` table. What must happen to `product_dim`?

Truth Is Normalised, Reporting Is Derived: Revealed

> The `product_dim` must be **regenerated** from the normalised source. The ETL pipeline rebuilds it, picking up the new name automatically.

If someone renamed it directly in `product_dim`, the star would show "Home & Living" while the OLTP system still showed "Home & Garden." That is **Failure 1 replicated between schemas**.

The safety argument holds only if the star is **always derived, never edited**.

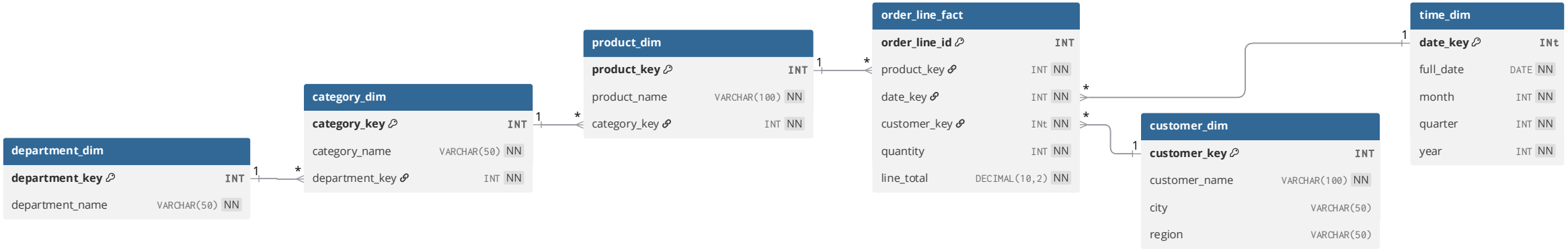
If you find yourself editing a dimension table directly — rather than regenerating it from the normalised source — you have lost the safety argument. Full stop.

Snowflake Schema: What If We Normalise Dimensions?

A colleague proposes normalising `product_dim` back into `product_dim → category_dim → department_dim`, each in BCNF. This is a **snowflake schema**.

```
Star:        order_line_fact ──▶ product_dim (flat)
             2 joins total for "revenue by department"

Snowflake:   order_line_fact ──▶ product_dim ──▶ category_dim ──▶ department_dim
             4 joins — the same chain from Failure 2
```



The snowflake reintroduces the **join chain** the star schema was designed to eliminate.

Three Schema Shapes Compared

	OLTP schema	Snowflake schema	Star schema
Dimension NF	BCNF	BCNF	2NF (not 3NF)
Purpose	Transactional truth	Reporting (normalised dims)	Reporting (flat dims)
Join depth	Full chain	Medium chain	Fact + dimension only
Redundancy	None	None	Deliberate in dimensions
Editable?	Yes	No (derived)	No (derived)

Since both snowflake and star are read-only and derived, update anomalies are not a risk in either. The star trades **storage redundancy** for **simpler queries**. In practice, if you are going to normalise the dimensions, you might as well query the OLTP source directly.

What Forces a Deliberate Downshift

BCNF is where you *start*. But some requirements demand controlled redundancy. Four categories:

Downshift	When	Safety mechanism
Snapshot	Preserve a historical value (shipping address at order time)	Immutability — set once, never updated
Derived attribute	Pre-compute a value for performance (`order_total`)	Transactional update — trigger or application logic
Reporting shape	Star schema for analytical workload	Read-only rebuild from normalised source
Dependency preservation	Decomposition would lose a business-rule constraint	Stay at 3NF; document the trade-off

Each downshift: state *what* NF you relax, *why*, and *how* consistency is maintained.

The Algorithm: Design to BCNF, Then Justify

Step 1: Verify each table against BCNF. List FDs, identify candidate keys, check every determinant. If a determinant is not a key → decompose.

Step 2: Identify requirements that demand controlled redundancy. Snapshots, derived attributes, reporting shapes, dependency preservation.

Step 3: Document each downshift. State the NF relaxed, the requirement, and the safety mechanism. An undocumented deviation is indistinguishable from a mistake.

Step 4: Annotate the schema. Mark each table's NF status and downshift type.

Recognition Checklist: Identify Normal Form

Given a table and its FDs, run these checks in order. Stop at the first violation:

Check	Question	If violated →
1NF	Are all values atomic? No repeating groups?	Not 1NF
2NF	(Only if composite key) Does every non-key attribute depend on the *whole* key?	1NF but not 2NF
3NF	Does any non-key attribute depend on another non-key attribute?	2NF but not 3NF
BCNF	Is every determinant a candidate key?	3NF but not BCNF

This is a checklist for **naming**, not a procedure for **building**. You design to BCNF; you use the ladder to diagnose legacy tables.

Starting Point: Topic 2's Eleven Tables

The ERD-to-schema translation from Topic 2 produced:

- `customers(customer_id, name, email, city)`
- `orders(order_id, customer_id, order_date, status)`
- `products(product_id, name, price, quantity_in_stock)`
- `order_lines(order_id, product_id, quantity, unit_price)` — composite PK
- `payments(payment_id, order_id UNIQUE, amount, payment_date, method)`
- `shipments(shipping_id, order_id UNIQUE, address, shipped_date, status)`
- `reviews(review_id, customer_id, product_id, rating, comment, review_date)`
- `suppliers(supplier_id, name, contact_email)`
- `supplier_products`, `categories`, `product_categories`

The ERD got the structure right. Does normalisation find anything the ERD missed?

Step 1: customers Has a Hidden Problem

`customers(customer_id, name, email, city)` — only determinant is `customer_id`. **BCNF.**

But real e-commerce systems store postcodes for shipping labels and tax. Adding `postcode` introduces an FD: `**postcode → city**`.

```
customers (with postcode added)
customer_id | name        | postcode | city
------------|-------------|----------|------------
1           | Alice Tan   | 018956   | Singapore
2           | Bob Patel   | 50088    | Kuala Lumpur
3           | Carol Lim   | 10110    | Bangkok
4           | Dave Wong   | 018956   | Singapore  ← same fact
```

`postcode` is a determinant but **not a candidate key** of `customers`. This violates BCNF — and it is exactly Failure 1's structural problem.

The Decomposition: Before and After

Before: `customers` stores city directly — redundant when two customers share a postcode.

After: extract `postcodes(postcode, city)` as a lookup table:

```
postcodes                          customers (city removed)
postcode | city                    customer_id | name        | postcode
─────────┼──────────               ────────────┼─────────────┼─────────
018956   | Singapore               1           | Alice Tan   | 018956
50088    | Kuala Lumpur             2           | Bob Patel   | 50088
10110    | Bangkok                  3           | Carol Lim   | 10110
                                    4           | Dave Wong   | 018956
```

City stored **once** per postcode. The decomposition is **lossless** — `postcode` is a candidate key of `postcodes`, so the join reconstructs exactly.

> **Predict:** After decomposition, how does a query retrieve a customer's city? Is this more or less convenient?

The Join Cost: Revealed

> The query joins `customers` to `postcodes` on `postcode`. **Less convenient** — one more join. But it eliminates truth-drift risk: update one row in `postcodes`, every customer at that postcode sees the correct city.

```
SELECT c.name, p.city
FROM   customers c
  JOIN postcodes p ON c.postcode = p.postcode;
```

Alice, Dave, and any future customer at `018956` all resolve through a **single `postcodes` row**. Update it once, and every customer's city is correct.

Cost: one extra join. **Benefit:** no truth drift. This is the structural cost of normalisation — the right cost for OLTP.

Step 1: The Remaining Tables

Table	Determinant(s)	Candidate key(s)	Status
`orders`, `products`	Single PK each	PK only	BCNF
`order_lines`	`{order_id, product_id}`	`{order_id, product_id}`	BCNF — no partial deps
`payments`	`payment_id`, `order_id` (UNIQUE)	Both	BCNF — two candidate keys
`shipments`	`shipping_id`, `order_id` (UNIQUE)	Both	BCNF
`suppliers`, `categories`	Single PK each	PK only	BCNF
Junction tables	Composite PK, no non-key attrs	PK	Trivially BCNF

Result: **eleven tables → twelve** — the only decomposition is `customers` → `customers` + `postcodes`.

> **Predict:** The `reviews` table has `(review_id, customer_id, product_id, rating, comment, review_date)`. List its FDs. Is it in BCNF?

> FDs: `review_id → customer_id, product_id, rating, comment, review_date`. Only determinant is the PK. **BCNF** — every non-key column describes *this review*.

Step 2: Justified Downshifts

Real-world shipping requires structured address fields. We extend `shipments` — replacing the generic `address` column with `shipping_address`, `shipping_city`, `shipping_postcode`.

Two requirements demand controlled departure from BCNF:

Downshift	Type	NF relaxed	Safety mechanism
`shipping_address`, `shipping_city`, `shipping_postcode` on `shipments`	**Snapshot**	BCNF (technically: `shipping_postcode → shipping_city`)	Immutable — set once at shipment time, never updated
`order_total` on `orders`	**Derived**	3NF (transitive: `order_id → order_lines → sum`)	Trigger-maintained — recalculated on every `order_lines` change

These are not violations we are "getting away with." They are **engineering decisions with named justifications**.

Snapshot: Shipping Address

```
shipments
shipping_id | order_id | shipping_address     | shipping_city | shipping_postcode
————————————|————————|————————————————|————————————|————————————
301         | 101      | 10 Raffles Quay      | Singapore     | 018956
302         | 102      | 10 Raffles Quay      | Singapore     | 018956
303         | 103      | 25 Jln Bukit Bintang | Kuala Lumpur  | 50088
```

If Alice later moves to Kuala Lumpur, rows 301–302 still show where the packages were **actually delivered**. The shipping address is a **historical fact**, not a copy of the current address.

The technical BCNF violation (`shipping_postcode → shipping_city`) cannot cause update anomalies because snapshot data is **immutable** — no updates ever occur.

Derived Attribute: Order Total

```
order_lines                                  orders
order_id | product_id | qty | unit_price      order_id | customer_id | order_total
---------+------------+-----+------------     ---------+-------------+------------
101      | 1          | 2   | 9.99           101      | 1 (Alice)   | 34.97
101      | 2          | 1   | 14.99          102      | 1 (Alice)   | 49.99
102      | 3          | 1   | 49.99
```

Order 101: `(2 × 9.99) + (1 × 14.99) = 34.97`. The truth lives in `order_lines`; `order_total` is a convenience copy kept safe by **transactional trigger maintenance**.

Cost: a controlled redundancy — `order_total` duplicates computable information. **Benefit:** instant display without re-aggregating line items.

Step 4: The Final OLTP Schema

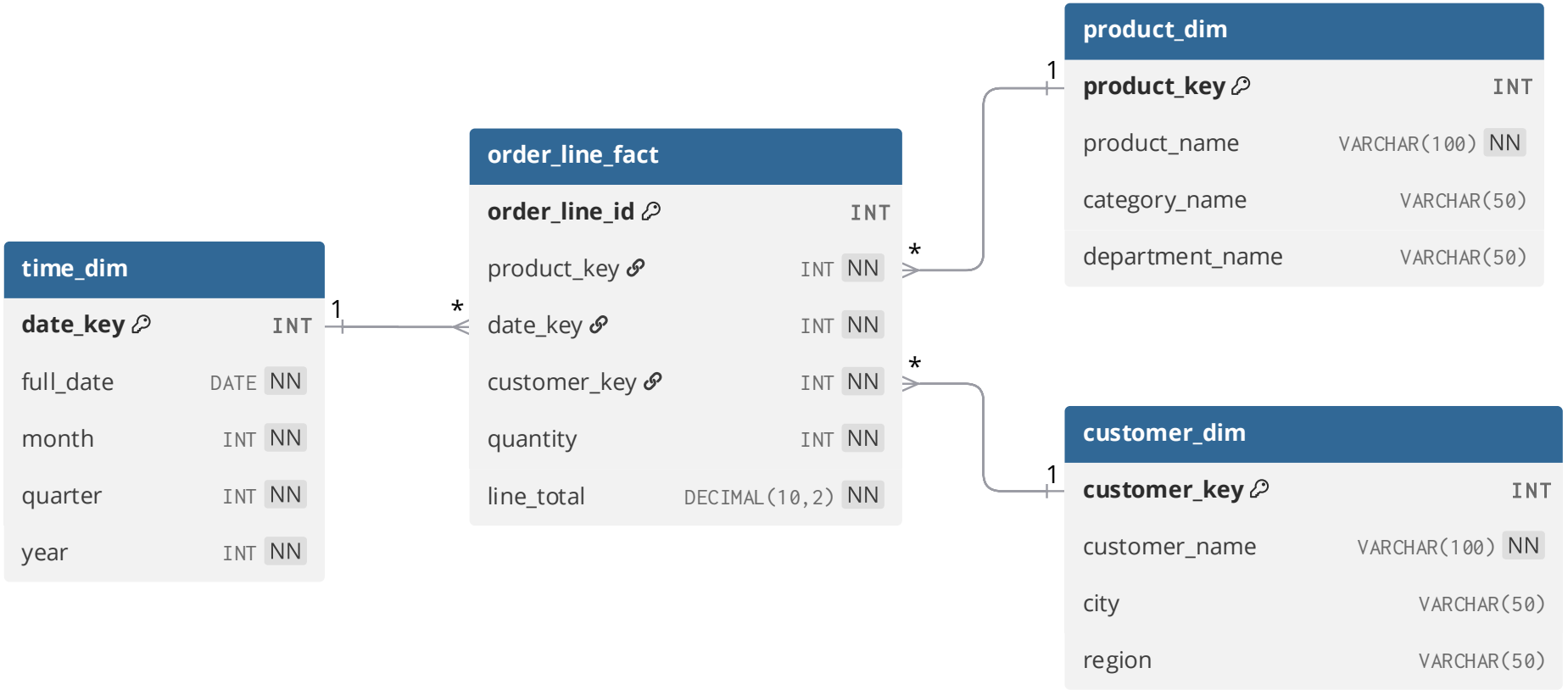
Twelve tables: eleven from Topic 2 + `postcodes` lookup from normalisation.

```
CREATE TABLE postcodes (
    postcode VARCHAR(10) PRIMARY KEY,
    city     VARCHAR(50) NOT NULL
);
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100) NOT NULL UNIQUE,
    postcode    VARCHAR(10) REFERENCES postcodes(postcode)
);
CREATE TABLE orders (
    order_id    INT PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES customers(customer_id),
    order_date  DATE NOT NULL,
    status      VARCHAR(20),
    order_total DECIMAL(10,2)  -- DERIVED: trigger-maintained
);
```

Key details: `REFERENCES postcodes(postcode)` enforces the decomposition. `UNIQUE` on `payments.order_id` makes it a candidate key. `order_total` is annotated as derived.

Building the Star Schema

Given the reporting requirement — "revenue by product category by month by customer region":



Star Schema: Sample Data

product_dim (3NF: category_name → department_name)

product_key	product_name	category_name	department_name
1	Widget A	Peripherals	Electronics
2	Widget B	Peripherals	Electronics
3	Gadget X	Peripherals	Electronics
4	Gadget Y	Peripherals	Electronics
5	Cable USB-C	Cables	Accessories

customer_dim (3NF: city → region)

customer_key	customer_name	city	region
1	Alice Tan	Singapore	Southeast Asia
2	Bob Patel	Kuala Lumpur	Southeast Asia
3	Carol Lim	Bangkok	Southeast Asia
4	Dave Wong	Singapore	Southeast Asia

order_line_fact (grain: one line item in one order)

order_line_id	product_key	date_key	customer_key	qty	line_total
1	1 (Widget A)	20240315	1 (Alice)	2	19.98
2	2 (Widget B)	20240315	1 (Alice)	1	14.99
3	3 (Gadget X)	20240620	1 (Alice)	1	49.99
4	1 (Widget A)	20240410	2 (Bob)	1	9.99
5	4 (Gadget Y)	20240722	3 (Carol)	1	79.99
6	2 (Widget B)	20240818	4 (Dave)	1	14.99

time_dim

date_key	full_date	month	quarter	year
20240315	2024-03-15	3	1	2024
20240410	2024-04-10	4	2	2024
20240620	2024-06-20	6	2	2024
20240722	2024-07-22	7	3	2024

"Revenue by department by quarter" needs only **two JOINs**: `fact → product_dim` + `fact → time_dim`. The BCNF equivalent needs **four** across five tables.

What Normalisation Does Not Solve

Normalisation eliminates redundancy from functional dependencies. It does not solve everything:

Limitation	What it means	Addressed
Higher NFs (4NF, 5NF)	Multi-valued and join dependencies — real but rare; BCNF handles the vast majority	Outside course scope
Non-redundancy performance	Missing indexes, bad queries, undersized hardware — *access-path* inefficiency, not *structural*	Topics 4–5
Star schema ETL	*How* to build and refresh the star — pipelines, slowly changing dimensions	Data engineering — beyond course

What normalisation *does* guarantee: if your OLTP schema is in BCNF, **no functional dependency can cause an update anomaly**.

Check 1: Truth-Drift Anomaly (Hospital)

A hospital stores patient city directly on the `patients` table: `(patient_id, name, postcode, city)`. Two patients share postcode `SW1A 1AA`. A borough is renamed. A developer updates one patient's city but not the other.

`SELECT DISTINCT city FROM patients WHERE postcode = 'SW1A 1AA'` returns:

> Two rows — the old name and the new. Truth has drifted. The FD `postcode → city` makes `postcode` a determinant that is not a candidate key — a **3NF violation** (transitive dependency). Decompose: `patients(patient_id, name, postcode)` + `postcodes(postcode, city)`. Update one row in `postcodes`, both patients see the correct city.

Check 2: Snapshot vs Redundancy (Pharmacy)

A pharmacy system stores `prescription_address` on the `dispensing` table. A colleague says: "The patient's address is already in `patients`. This is redundant — remove it and join."

Are they correct?

> **No.** The `prescription_address` records where the medication was delivered at dispensing time — a **historical fact**. If the patient moves, the dispensing record must still show the original delivery location. This is a **snapshot**, not a copy. The technical BCNF violation is safe because the data is immutable. Removing it would **destroy historical data**.

Check 3: Star Schema NF Identification (Music Streaming)

A music streaming service has `artist_dim(artist_key, artist_name, genre_name, label_name)`. Each genre belongs to one label. Identify the FDs and state the normal form.

> FDs: `artist_key → artist_name, genre_name, label_name` (key determines all). Also: `genre_name → label_name` (each genre belongs to one label). The determinant `genre_name` is not a candidate key. The table is in **2NF but not 3NF** — transitive dependency. In the star schema: acceptable (read-only, derived).

Checks 1–3: Pattern So Far

Check	Skill tested	Key move
1 — Hospital	**Detection** — spotting the anomaly	Determinant not a key → decompose
2 — Pharmacy	**Judgement** — redundancy vs history	Same structure, opposite diagnosis — mutability decides
3 — Music streaming	**Precision** — naming the exact NF	2NF not 3NF; acceptable in read-only context

Three skills so far: detect a violation, judge whether it matters, name it precisely. The next three checks extend to **cost**, **transfer**, and **boundary**.

Check 4: Reporting Query Comparison (Tax Authority)

A tax authority needs "total revenue by industry by fiscal year." The BCNF schema has: `filings → taxpayers → businesses → industries`, plus `filings → fiscal_periods` for the year. The star schema has: `filing_fact → industry_dim` and `filing_fact → time_dim`.

Count the JOINs in each version.

> **BCNF:** four JOINs across five tables. **Star:** two JOINs. The star is structurally simpler because dimension tables pre-flatten the join chain. Both produce the same totals — the star is derived from the same data.

Check 5: Partial Dependency (Ride-Sharing)

A ride-sharing company has `trip_assignments(driver_id, vehicle_id, driver_name, vehicle_plate, trip_date)` with composite PK `(driver_id, vehicle_id, trip_date)`.

`driver_name` depends only on `driver_id`. `vehicle_plate` depends only on `vehicle_id`.

What normal form is this table in? What is the violating FD?

> **1NF but not 2NF.** The violating FDs are partial dependencies: `driver_id → driver_name` and `vehicle_id → vehicle_plate` — non-key attributes depend on part of the composite key, not the whole key. Fix: decompose into `drivers(driver_id, driver_name)`, `vehicles(vehicle_id, vehicle_plate)`, `trip_assignments(driver_id, vehicle_id, trip_date)`.

Check 6: Cross-Topic: Design-Time vs Runtime

Topic 1 placed the relational model at "engine rejects" on the enforcement spectrum. Normalisation adds a design-time discipline on top of the engine's runtime enforcement.

Can the database engine enforce BCNF? Can you write a constraint that says "every determinant must be a candidate key"?

> **No.** BCNF is a design principle, not a runtime constraint. The engine enforces the *consequences* of BCNF — PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE — but it cannot verify that the designer correctly identified all FDs. The engine enforces constraints **within** a schema; normalisation determines the **shape** of the schema. Two disciplines, two moments — complementary.

Checks 1–6: The Arc

Check	Domain	What it tests	Lesson
1 — Truth drift	Hospital	Detection: spotting the anomaly	Redundancy → drift; decompose along the violating FD
2 — Snapshot	Pharmacy	Judgement: redundancy vs history	Same structure, opposite diagnosis — mutability decides
3 — Star NF	Music streaming	Precision: naming the exact NF	2NF not 3NF; acceptable in read-only derived context
4 — Query comparison	Tax authority	Cost: counting the structural difference	Star pre-flattens; 2 joins vs 4
5 — Partial dep	Ride-sharing	Transfer: vocabulary in a new domain	Composite key + partial dependencies → 2NF violation
6 — Design vs runtime	Cross-topic	Boundary: where normalisation stops	BCNF is design-time; engine enforces within the shape

Progression: **detection → judgement → precision → cost → transfer → boundary**.

Callback: Revisiting the Two Failures

Earlier we asked: are the two failures contradictory?

Failure 1 — the city that drifted. The FD `postcode → city` made `postcode` a determinant that was not a candidate key — a 3NF violation. The algorithm's Step 1 catches this: decompose. After decomposition, city stored once, both tables BCNF.

Failure 2 — the dashboard nobody used. The schema was in BCNF; the problem was shape mismatch. The star schema resolves it: two joins instead of four. 3NF violations in dimensions are safe because the star is read-only and derived.

Both failures are now **diagnosable with precision**. The vocabulary does not just describe the problem — it points directly to the fix.

Course Map

```
Topic 1   Introduction to Data Models
Topic 2   Conceptual Modelling (ERD → relational schema)
    │
    ├── ▸ Topic 3  Normalisation (BCNF + star schema)  ← you are here
    │
    ├── Topic 4    SQL Querying (joins, aggregation, CTEs)
    └── Topic 5    Enforcement (constraints, transactions, indexes)
         ──────── relational foundation complete ────────
Topic 6   Document Model (JSON, MongoDB)
Topic 7   XML / XSD
Topics 8–9  RDF & SPARQL
Topic 10  Property Graph Model
Topic 11  Capstone: Multi-Model Architecture
```

The BCNF schema produced here is the input for Topics 4 (SQL queries) and 5 (enforcement hardening).

Concepts That Recur

Concept	Introduced here as	Fully developed in
Snapshot vs truth copy	Snapshot columns on `shipments` (immutable historical fact)	Topic 6 (embedded `subscriptionSnapshot` in document model)
Derived attribute	`order_total` maintained by trigger	Topic 5 (implemented as a trigger with transactional consistency)
Star schema / OLAP shape	Denormalised dimension tables for reporting	Topic 11 (multi-model: which workload gets which shape)
Lossless decomposition	Split and join to reconstruct — safety test	Topic 6 (embed vs reference: can you reconstruct the normalised view?)
Shape alignment within one model	Two relational shapes for two workloads	Topic 6 (aggregate design), Topic 11 (model-per-workload)
Design-time vs runtime	BCNF is design; engine enforces within	Topic 5 (constraints enforce BCNF's decisions), Topic 7 (XSD as design + runtime contract)

The snapshot concept transfers directly: "is this embedded data a historical fact or a copy that must be kept in sync?"

Lab Preview

Three-part lab — diagnosis, BCNF design, and star schema comparison.

- **Part 1 — Identify normal form exercises:** Four tables at different NF levels. For each: state the current NF, the violating FD, and the decomposition.
- **Part 2 — Live BCNF design (dbdiagram.io → MySQL):** Build the BCNF core schema for Orders & Payments. Verify each table against BCNF. Add one justified downshift (snapshot or derived) with documentation.
- **Part 3 — Star schema for reporting:** Design a star schema from the BCNF core. Load pre-built data, run "revenue by department by quarter" against both schemas, compare execution times.

Lab file: `03-lab-normalisation.ipynb`